מעגלים משולבים ומבוא ל-VLSI

עבודה מס׳ 2 - תכנון, מידול וסימולוציה של רכיבים ב־Verilog-A

מגיש : אריאל פיסחוב

תאריך הגשה : 6.6.26

תוכן עניינים

חלק 1 - רכיב Sample and hold (S/H)

1.1 - קוד ב-VerilogA עבור רכיב זה 3
1.2 - יצירת סימבול עבור הרכיב 4
1.3 - Testbench לרכיב 4
1.4 - תוצאות 5

חלק 2 - רכיב Comparator

2.1 - קוד ב-VerilogA עבור רכיב זה 7
2.2 - יצירת סימבול עבור הרכיב 7
2.3 - Testbench לרכיב 8
2.4 - תוצאות 9

חלק 3.1 - רכיב DAC

3.1.1 - קוד ב-VerilogA עבור רכיב זה 10
3.1.2 - יצירת סימבול עבור הרכיב 11
3.1.3 - Testbench לרכיב 12
3.1.4 - תוצאות 13

חלק 3.2 - רכיב DAC Reference

3.2.1 - קוד ב-VerilogA עבור רכיב זה 16
3.2.2 - יצירת סימבול עבור הרכיב 18
3.2.3 - Testbench לרכיב 18
3.2.4 - תוצאות 19

בעבודה הבאה, נתכנן מספר רכיבים בסיסיים ונבדוק את עבודתם. נגדיר לשם כך את V_{DD}:

$$V_{DD} = 1.5 + 6 \cdot 10^{-3} \cdot G = 1.5 + 6 \cdot 10^{-3} \cdot 5 = 1.53V$$

חלק 1 - רכיב Sample and hold (S/H)

רכיב H/S הינו רכיב שדוגם באופן רגעי את האות האנלוגי כדי לשמור על ערך קבוע עבור השהיית האות מספר מחזורי שעון (אם הוא ימשיך להשתנות המוצא יהיה שגוי).

1.1 - קוד ב-VerilogA עבור רכיב זה

בסעיף זה, תחת הספריה של תרגיל הבית שלנו בתוכנה, נפתח cellview חדש מסוג VerilogA. נכתוב את הקוד עבור רכיב Sample and Hold.

הקוד:

```
1  // VerilogA for HW3_GalAriel, Sample&Hold, veriloga
2
3  `include "constants.vams"
4  `include "disciplines.vams"
5
6  module Sample_and_hold(out, in, clk, vdd, gnd);
7
8      output out;
9      input in,clk,vdd,gnd;
10     electrical out,in,clk,vdd,gnd;
11     real  Vhold;
12
13     analog begin
14
15           @(cross(V(clk) - V(vdd)/2, +1))
16             Vhold = V(in);
17
18         V(out) <+ transition(Vhold,0,1n);
19     end
20
21 endmodule
```

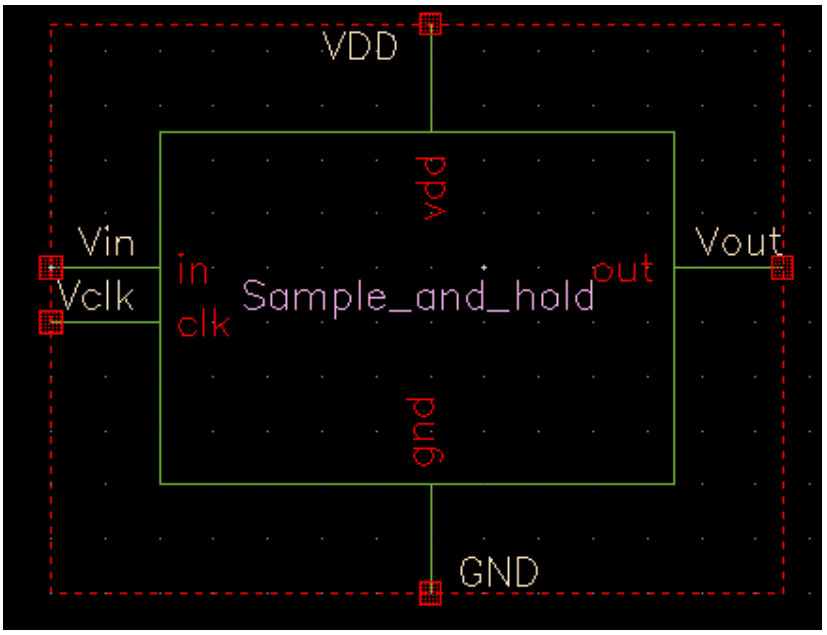
אופן פעולת הקוד: בתחילת הקוד מוגדר מודול בשם Sample_and_hold, שתפקידו לדגום את מתח הכניסה ולשמור אותו ביציאה. המודול כולל את הכניסות in, clk, vdd, gnd ואת היציאה out. בנוסף מוגדר משתנה פנימי בשם Vhold, שבו נשמר ערך המתח שנדגם. כלומר, משתנה זה משמש כזיכרון פנימי של המודול, ושומר את הערך האחרון של מתח הכניסה שנמדד בזמן המתאים.

בתוך הבלוק analog begin נעשה שימוש בפקודת @cross, שמזהה מתי אות השעון clk חוצה את הערך $V_{DD}/2$ כלפי מעלה. ברגע שחציה זו מתרחשת, הקוד דוגם את מתח הכניסה V_{in} ושומר אותו בתוך V_{hold}. לאחר מכן, מתח היציאה V_{out} מקבל באופן רציף את הערך השמור ב־V_{hold}, ולכן היציאה נשארת קבועה גם אם מתח הכניסה משתנה, עד לרגע הדגימה הבא בעליית השעון.

1.2 - יצירת סימבול עבור הרכיב

לאחר שבדקנו שהקוד עובד כראוי, יצרנו את הסימבול המתאים עבור הרכיב S/H.

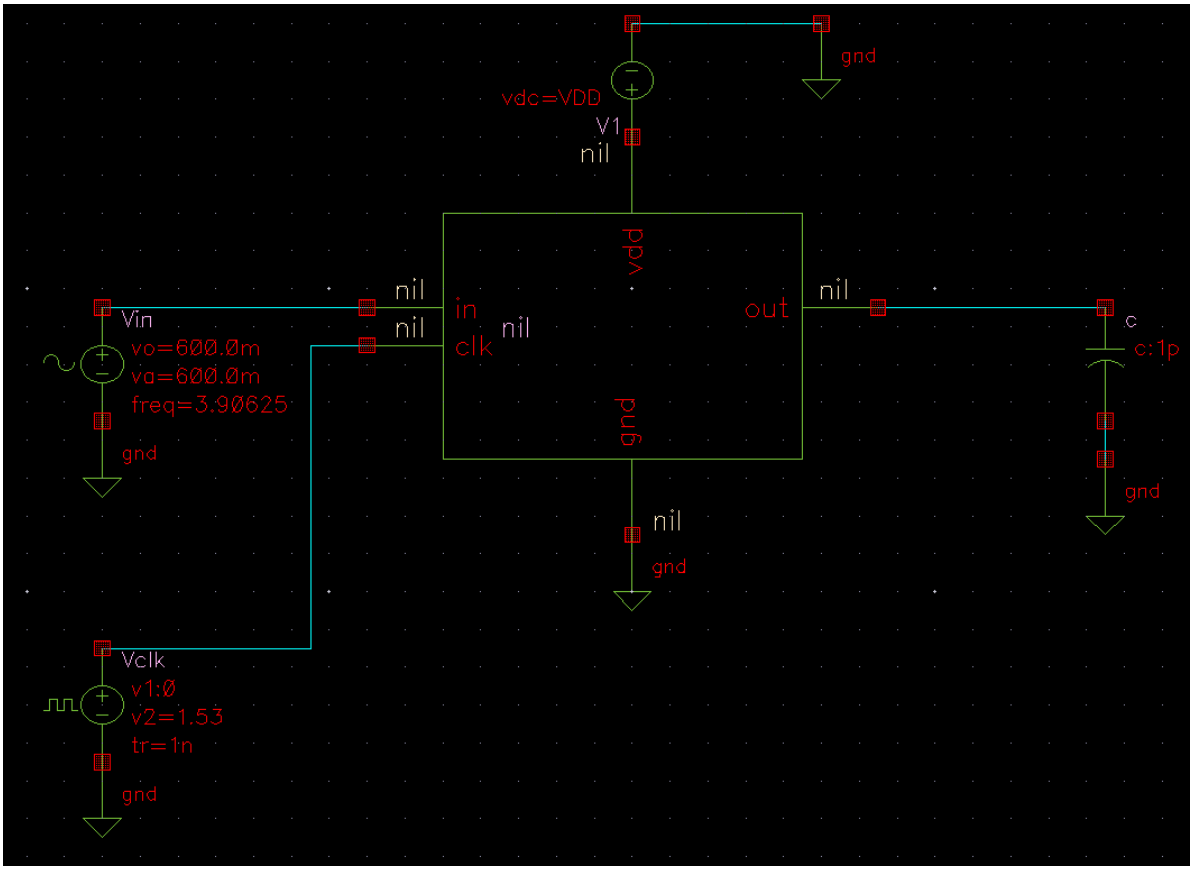
הסימבול:



1.3+1.4 - Testbench לרכיב

בסעיף זה נעשה TB לרכיב על מנת לבדוק את פעולותיו, ונציג את V_{in}, V_{out}, V_{clk}.

הסכמה לTB:

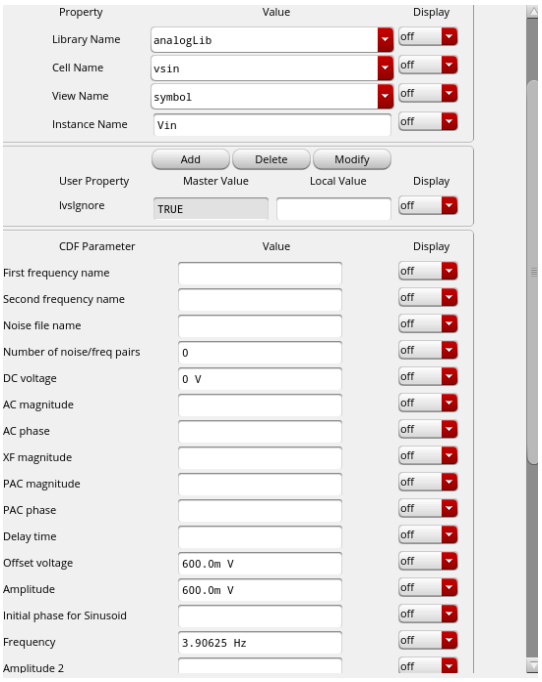


לצורך הTB, נקבע לפי הנחיות התרגיל את ערכי הרכיבים.

עבור מתח הכניסה V_{in}:

$$freq = fin = n_{period} * \left(\frac{f_s}{n_{samples}}\right), V_{in} = 1.2V, V_{offset} = V_{amplitude} = \frac{V_{in}}{2}$$

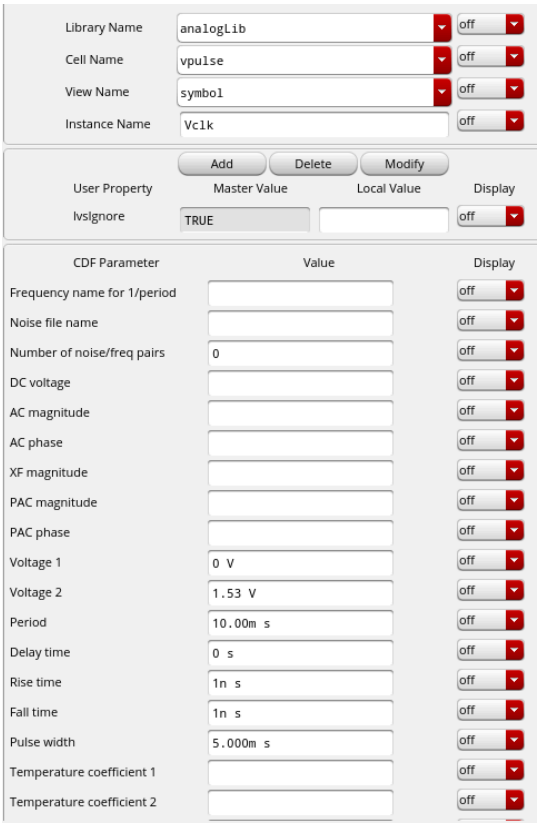
כאשר: $n_{period} = 5; n_{samples} = 2^7; f_s = 100Hz$



$$freq = fin = 5 \cdot \left(\frac{100Hz}{2^7}\right) = 3.90625Hz$$

$$V_{offset} = V_{amplitude} = 0.6V$$

עבור מתח השעון $V_{clk} = V_{pulse}$: clk (כמתח גם) מסוג Pulse בין 0 לVDD עם $\text{period} = \frac{1}{f_s}$.



הפולס הוא בין 0 ל-$V_{DD} = 1.53V$, ו-

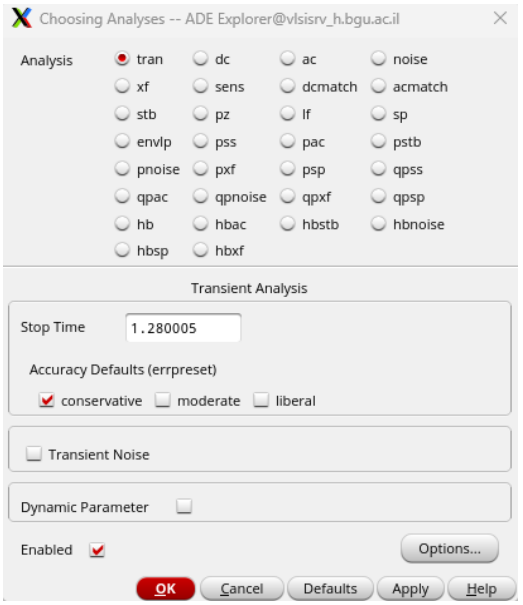
$Period = \frac{1}{100Hz} = 10ms$.

את רוחב הפולס נגדיר כחצי זמן המחזור.

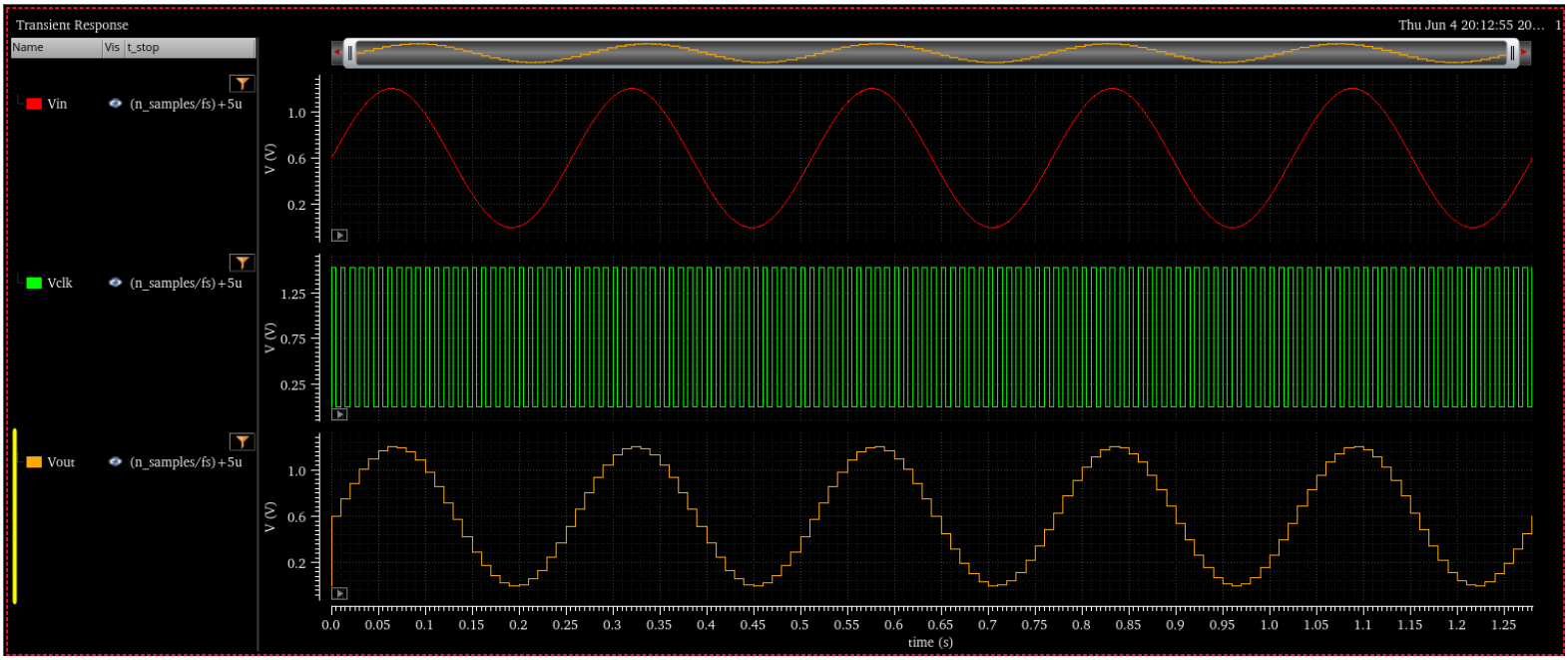
עבור מתח היציאה V_{out}, נשים קבל במוצא של $C = 1pF$.

כעת נריץ סימולציית tran מ-0 עד $VAR("t_{stop}") =$ כאשר $t_{stop} = \frac{n_{samples}}{f_s} + 5u$, כאשר

ערכו של t_{stop} מחושב והוכנס לאנליזה מטה:



התוצאות:



כפי שניתן לראות, הרכיב לוקח דגימה של אות הכניסה הסינוסי (באדום) בכל עלייה של השעון (בירוק), ושומר את הערך הזה במוצא (בצהוב) עד הדגימה הבאה. לכן מתקבל במוצא אות בצורת מדרגות, שדומה בצורה טובה לאות הסינוס המקורי. כלומר הרכיב עובר כראוי.

חלק 2 - רכיב Comparator

רכיב Comparator הינו רכיב משווה בין האות האנלוגי לאות רפרנס שאותו הוא מקבל או ממוצא של רכיב אחר או מהגדרת ערך V_{ref}.

הפלט שלו יהיה גבוה אם האות המקורי גדול יותר מ V_{ref} ונמוך אחרת.

2.1 - קוד ב-VerilogA עבור רכיב זה

הקוד:

```
// VerilogA for HW3_GalAriel, Comparator, verilog

`include "constants.vams"
`include "disciplines.vams"

module Comparator(V_sin, V_ref, V_out, VDD, GND);

input V_sin, V_ref;
output V_out;

electrical V_sin, V_ref, V_out, VDD, GND;

analog begin

    V(V_out) <+ transition((V(V_sin) > V(V_ref)) * V(VDD), 0, 1n);

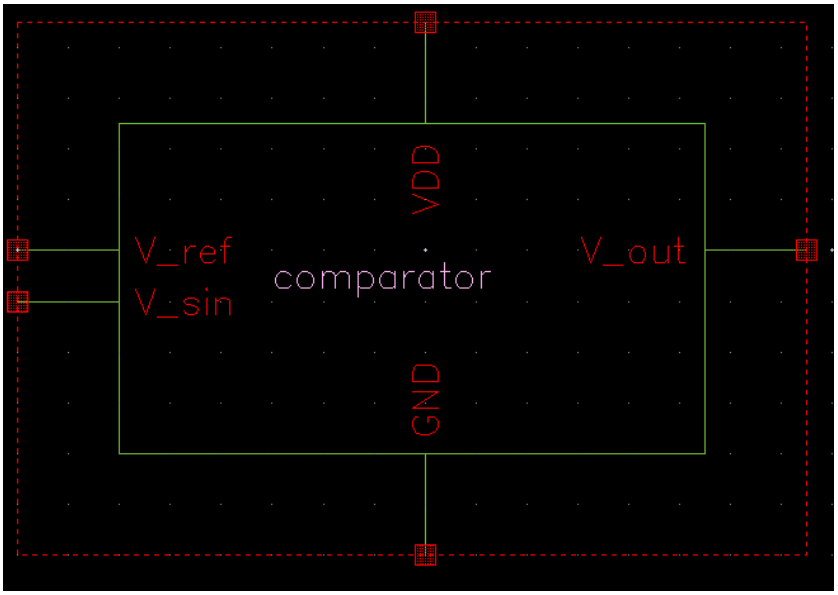
end

endmodule
```

אופן פעולת הקוד: הקוד משווה בין המתח V_{sin} לבין מתח הייחוס V_{ref}. אם V_{sin} גדול מ־ V_{ref}, היציאה V_{out} מקבלת את ערך המתח V_{DD}, ואם לא — היא יורדת ל־0. הפונקציה transition גורמת למעבר של היציאה להיות חלק יותר ולא קפיצה חדה לגמרי, עם זמן מעבר של 1n.

2.2 - יצירת סימבול עבור הרכיב

הסימבול:



Testbench - 2.3+2.4 לרכיב

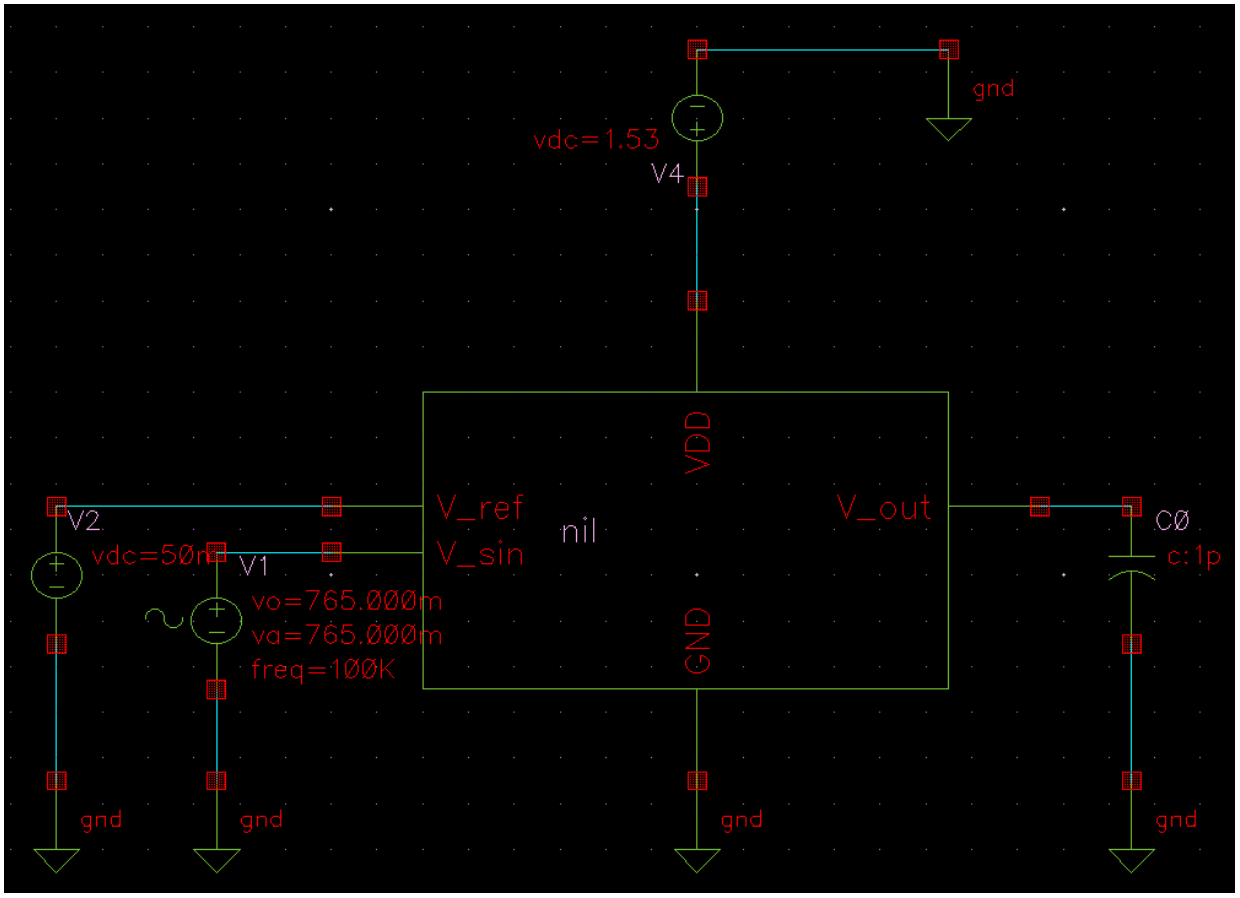
בסעיף זה נעשה TB לרכיב על מנת לבדוק את פעולותיו, ונציג את V_{ref}, V_{out}, V_{in}, כאשר

V_{ref} אצלנו: $V_{ref} = G * 10 * 10^{-3} = 50mV$ (G=5 מספר הקבוצה).

נגדיר בנוסף את מתח כניסה מסוג סינוסי כאשר $V_{offset} = V_{amplitude} = \frac{V_{in}}{2}$, ו- $V_{in} = V_{DD}$

עם תדר $f = 100kHz$.

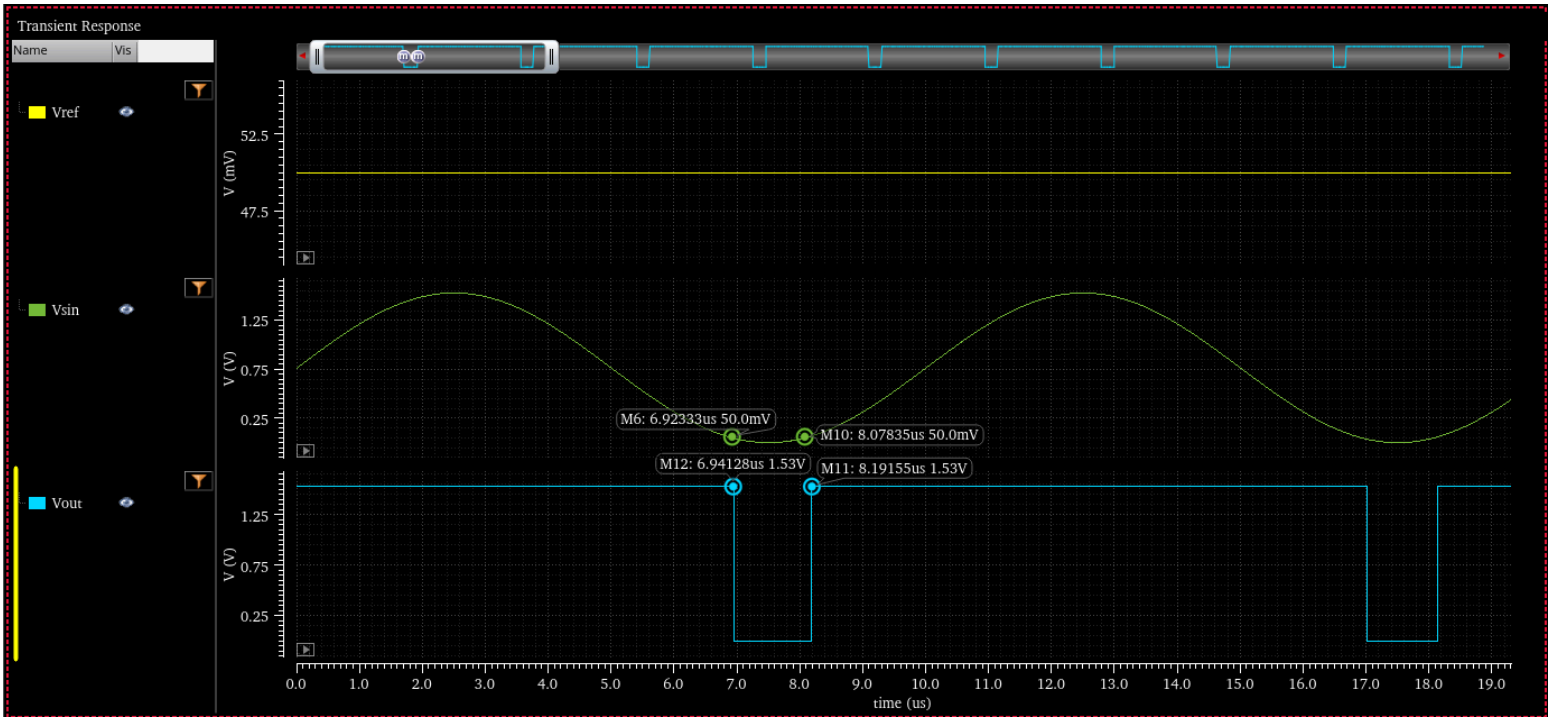
<u>הסכמה לTB:</u>



לצורך הTB, נקבע לפי הנחיות התרגיל את ערכי הרכיבים במערכת, באופן דומה לסעיף הקודם.

נריץ סימולציית trans שתכלול לפחות 2 מחזורים של הכניסה והמוצא.

התוצאות:



לאחר ביצוע הסימולציה ניתן לראות כי קיבלנו את הנדרש. כאשר מתח הכניסה הסינוסי (בירוק) יורד מתחת ל-$V_{ref} = 50mV$ (בצהוב), המוצא (בכחול) יורד ל0- לוגי, וכאשר הכניסה עולה מעל $V_{ref} = 50mV$, המוצא חוזר ל-1 לוגי. בנוסף, ניתן לראות כי ישנו הפרש מזערי בעקבות ה -transition שלוקח זמן למתח המוצא להשתנות מ-0 לוגי ל-1 לוגי למרות שמתח הכניסה כבר השתנה ביחס לV_{ref}. לדוגמה בשינוי בראשון, ניתן לראות שמתח המוצא משתנה מV_{DD} ל-0 בדילי של 18ns, כלומר 18ns אחרי שV_{sin} חצה את הערך $V_{ref} = 50mV$ בירידה. אותו הדבר ניתן לראות בעליה, ששם ישנו הפרש של 115ns.

תופעה זו מתרחשת, (בנוסף לשימוש בפונקציית הtransition) מכיוון שהמעגל אינו אידיאלי, ולכן המוצא לא יכול להשתנות באופן מיידי. למעגל יש זמן תגובה סופי, הנובע למשל מקיבולים פנימיים, זמן מעבר של הטרנזיסטורים, וקצב שינוי מוגבל של המתח. לכן, גם אחרי שהכניסה כבר עברה את הסף, לוקח זמן קצר עד שהמוצא מזהה את השינוי ומבצע את המעבר הלוגי מ-0 ל-1 או מ-1 ל-0.

בנוסף, ניתן לראות שזמן המעבר בעלייה שונה מזמן המעבר בירידה. זה קורה כי המעגל אינו אידיאלי ואינו סימטרי לגמרי. טעינת הקיבול במוצא לכיוון VDD ופריקתו לכיוון האדמה נעשות דרך מסלולים שונים במעגל. לכן המוצא יכול לעלות בקצב אחד ולרדת בקצב אחר, ולכן מתקבל דילי שונה בין המעברים.

לסיכום, תוצאות הסימולציה תואמות את התיאוריה, והרכיב מתפקד כמצופה. ניתן לראות שמתח המוצא משתנה בהתאם לחציית מתח הייחוס על ידי מתח הכניסה, כאשר קיימים זמני מעבר סופיים ושונים בין עלייה לירידה, כפי שמצופה מרכיב לא אידיאלי.

חלק 3.1 - רכיב (DAC)

רכיב ה DAC מקבל אות דיגיטלי וממיר אותו לאות אנלוגי.

3.1.1 - קוד ב-VerilogA עבור רכיב זה

הקוד:

```
// VerilogA for HW3_GalAriel, DAC, veriloga

`include "constants.vams"
`include "disciplines.vams"

module DAC(out, in, clk, vdd, gnd);

    output out;
    input [7:0] in;
    input clk, vdd, gnd;

    electrical [7:0] in;
    electrical out, clk, vdd, gnd;

    real out_a, out_b;
    real in_vals[7:0];

    integer i;
    genvar j;

    analog begin

        @(initial_step) begin
            out_a = 0.5 * V(vdd);
            out_b = 0.5 * V(vdd);
            i = 0;
        end

        @(cross(V(clk) - 0.5 * V(vdd), +1)) begin


            for (j = 0; j < 8; j = j + 1) begin

                in_vals[j] = V(in[j]);
            end

            if (i < 8) begin

                out_a = out_a + 0.5 * in_vals[i] * pow(2, i);
                out_b = out_b - 0.5 * in_vals[i] * pow(2, i);
                i = i + 1;
            end
        end

        V(out) <+ transition((out_a - out_b) / (V(vdd) * 256), 0, 1n);
    end

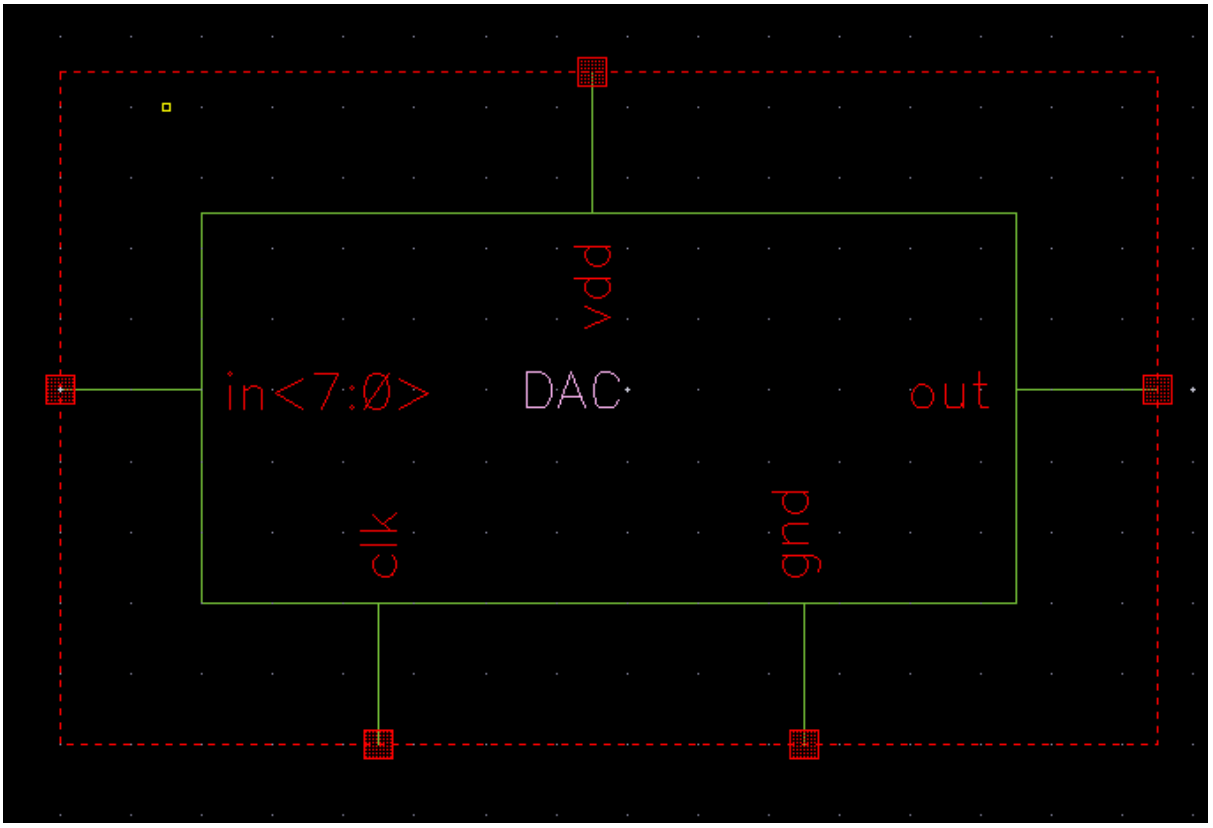
endmodule
```

<u>אופן פעולת הקוד</u>: המודול מקבל כניסה דיגיטלית של 8 ביטים, אות שעון clk, מתח אספקה V_{DD}, ומוציא אות אנלוגי דרך out. בתחילת הסימולציה, בתוך @(initial_step), הקוד מאתחל את שני המשתנים הפנימיים out_a ו־out_b לערך של חצי ממתח האספקה, כלומר $V_{DD}/2$. בנוסף, המשתנה i מאותחל ל־0, כדי להתחיל לקרוא את הביטים מהביט הראשון. המערכת מתחילה ממצב מאוזן, שבו out_a ו־out_b שווים, ולכן ההפרש ביניהם הוא אפס.

לאחר מכן, בכל פעם שאות השעון clk חוצה כלפי מעלה את $V_{DD}/2$, כלומר בעליית שעון, הקוד קורא את כל ערכי הביטים מהכניסה ושומר אותם במערך in_vals. אחר כך, כל עוד $i < 8$, הקוד לוקח בכל מחזור שעון ביט אחד, מכפיל אותו במשקל המתאים שלו לפי המיקום שלו, כלומר i^2, ומעדכן את שני המשתנים out_a ו־out_b: הוא מוסיף את התרומה ל־out_a ומחסיר אותה מ־out_b. בצורה זו נוצרת יציאה דיפרנציאלית, שבה צד אחד עולה והשני יורד. בסוף, היציאה out מקבלת את ההפרש out_a - out_b, מנורמל לפי $256*V_{DD}$, משום שמדובר ב־DAC של 8 ביטים עם $2^8 = 256$ רמות. לבסוף, השתמשנו בדומה לסעיפים הקודמים בפונקציה transition הגורמת לשינוי ביציאה להיות הדרגתי וחלק יותר, עם זמן מעבר של 1ns, במקום קפיצה מיידית.

3.1.2- יצירת סימבול עבור הרכיב

<u>הסימבול</u>:



Testbench -3.1.3 + 3.1.4 לרכיב

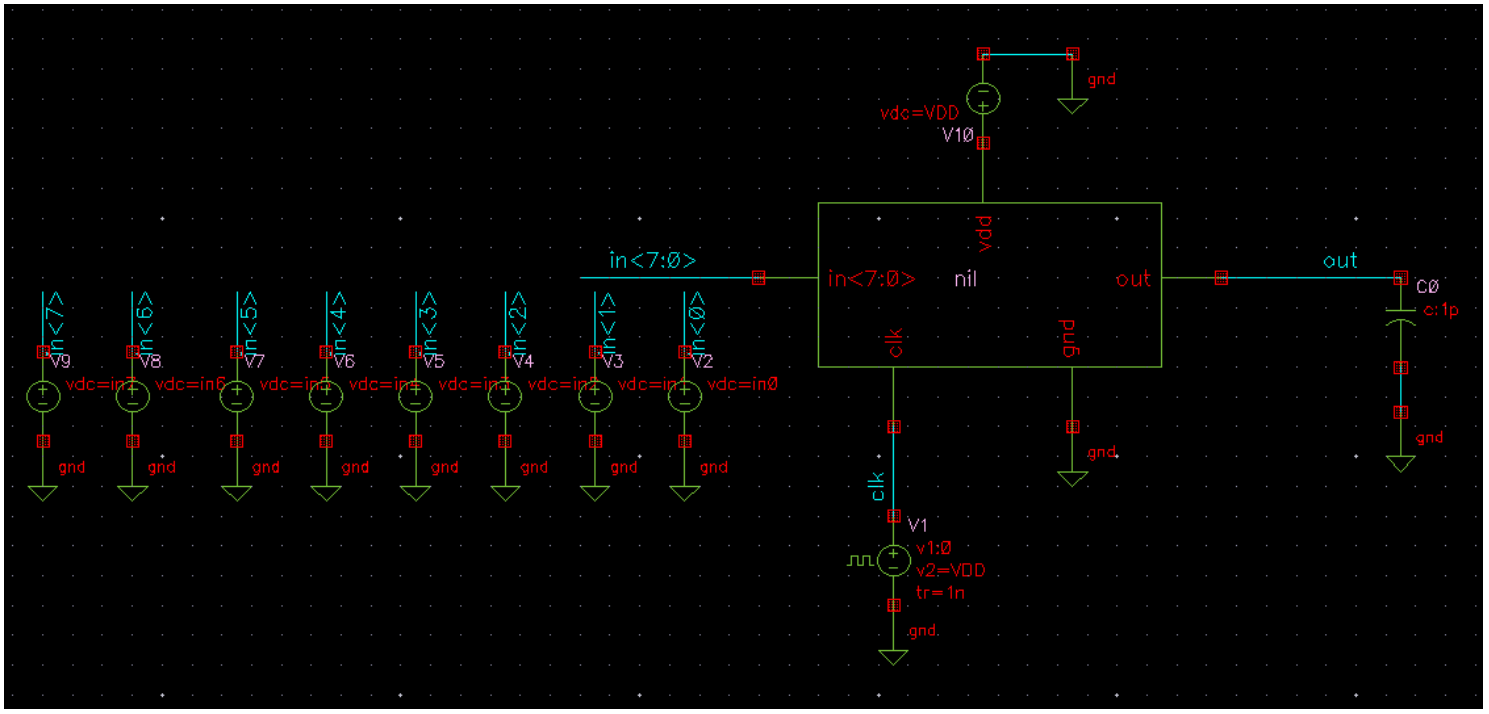
ניצור את ה-TB באופן הבא:

בכניסה נכניס מתחי DC עם הערכים הבאים בהתאם:

1) הכניסה תהיה מספר הקבוצה שלנו - 5
2) הכניסה תהיה המספר ההופכי לקבוצה שלנו (כלומר כאשר הספרות הפוכות).

במוצאים נשים קבלים בגודל $C = 1pF$. נגדיר clk (כמתח גם) מסוג pulse בין 0 ל V_{DD} עם $Period = \frac{1}{f_s}$, כאשר f_s הוא כמו בשאלה 1, ונבדוק אם ההפרש הדיפרנציאלי בין המוצאים תואם את הערך אותו אנו רוצים לקבל (כלומר את מספר הקבוצה שלנו 5, המנורמל ל256 ביטים).

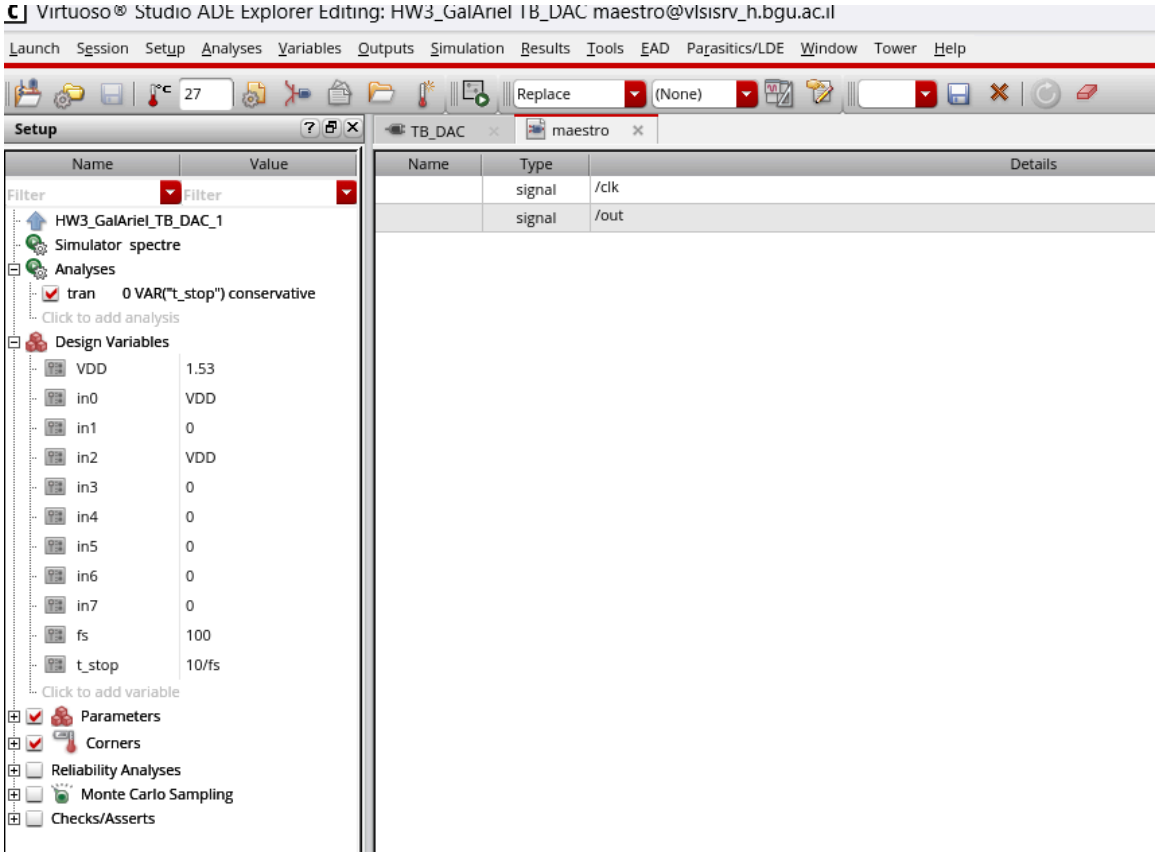
הסכמה לTB:



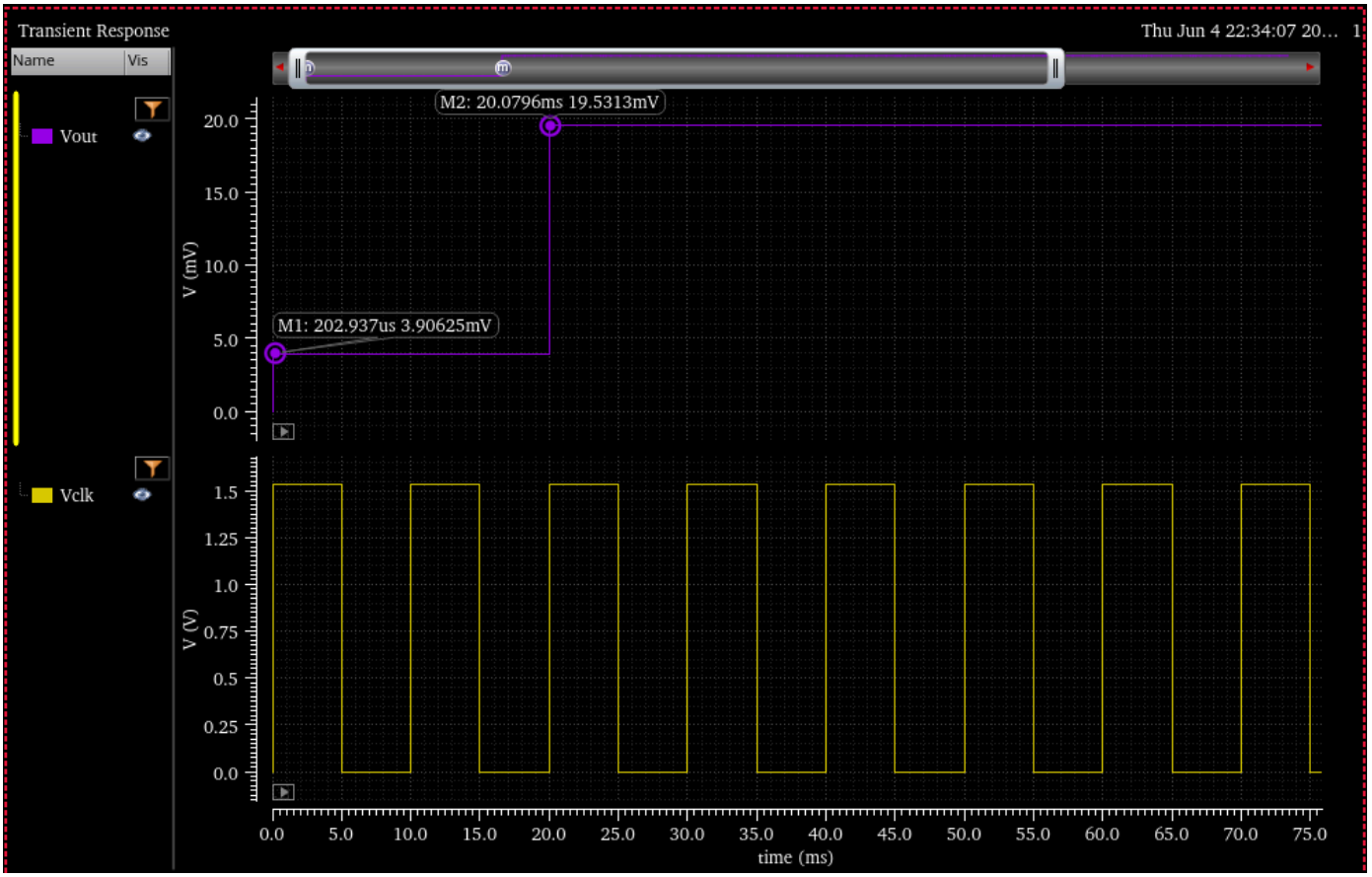
כעת נריץ את הסימולציות עבור שני המקרים שתיארנו מעלה, ונדון בתוצאות.

1) עבור כניסה 5 (כמספר הקבוצה שלנו) - 00000101

הגדרנו סימולציה עם הערכים הרלוונטיים לכל כניסה בהתאם למספר הקבוצה שלנו:



תוצאות:

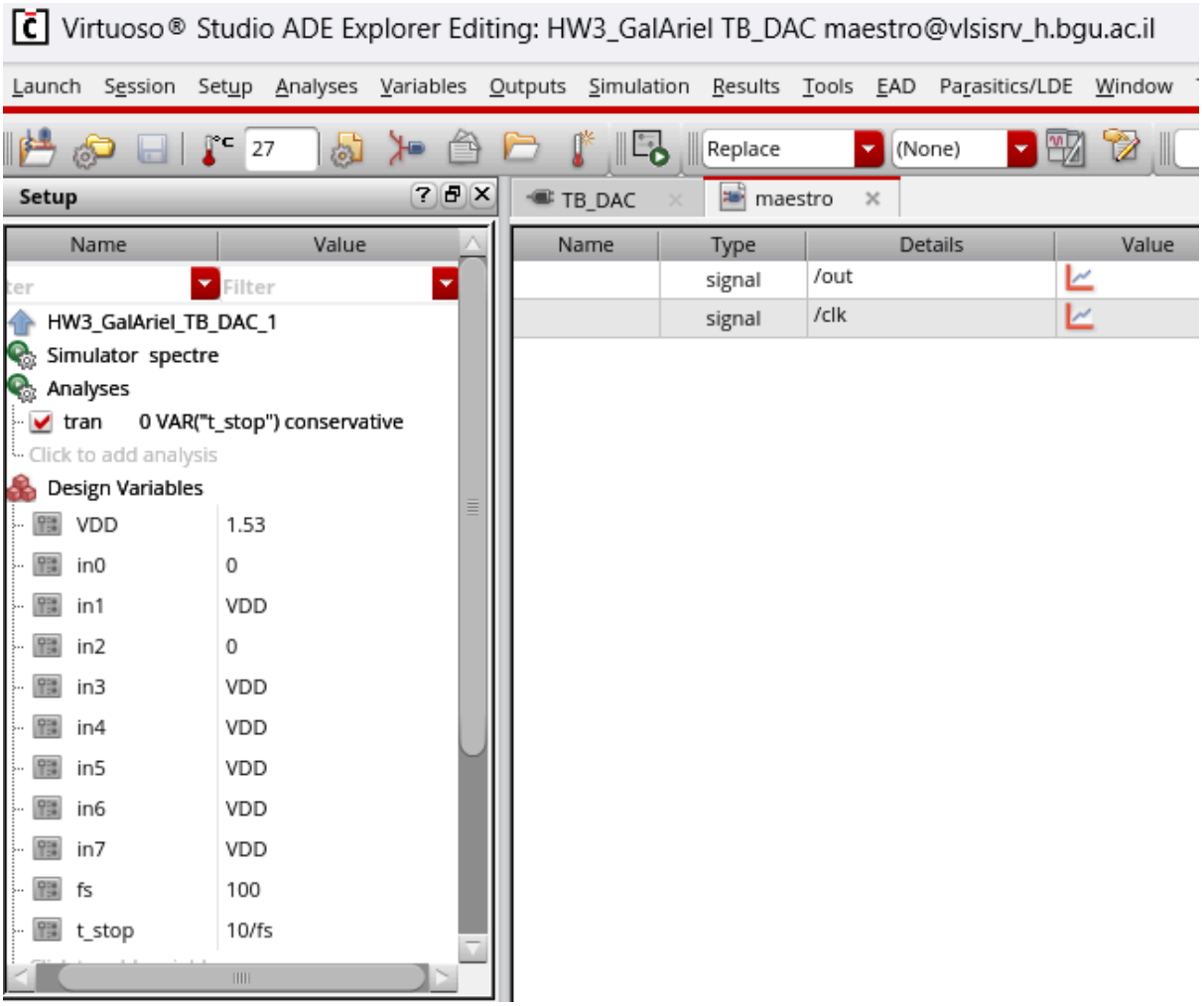


ניתן לראות מהגרף, כצפוי, שמתרחשת עליה במוצא (בסגול) בעליות השעון (בצהוב) הראשונה והשלישית, כלומר המתאימות לקריאת הביטים במקומות 0 ו-2, שהן למעשה היחידות שנמצאות ב-1 לוגי במקרה זה.
כעת נבדוק האם ההפרש הדיפרנציאלי בין המוצאים תואם את הערך אנו רוצים לקבל, כלומר לקבל את מספר הקבוצה מנורמל ל256 ביטים. נראה אם החישוב אכן תואם לערכים בגרף:

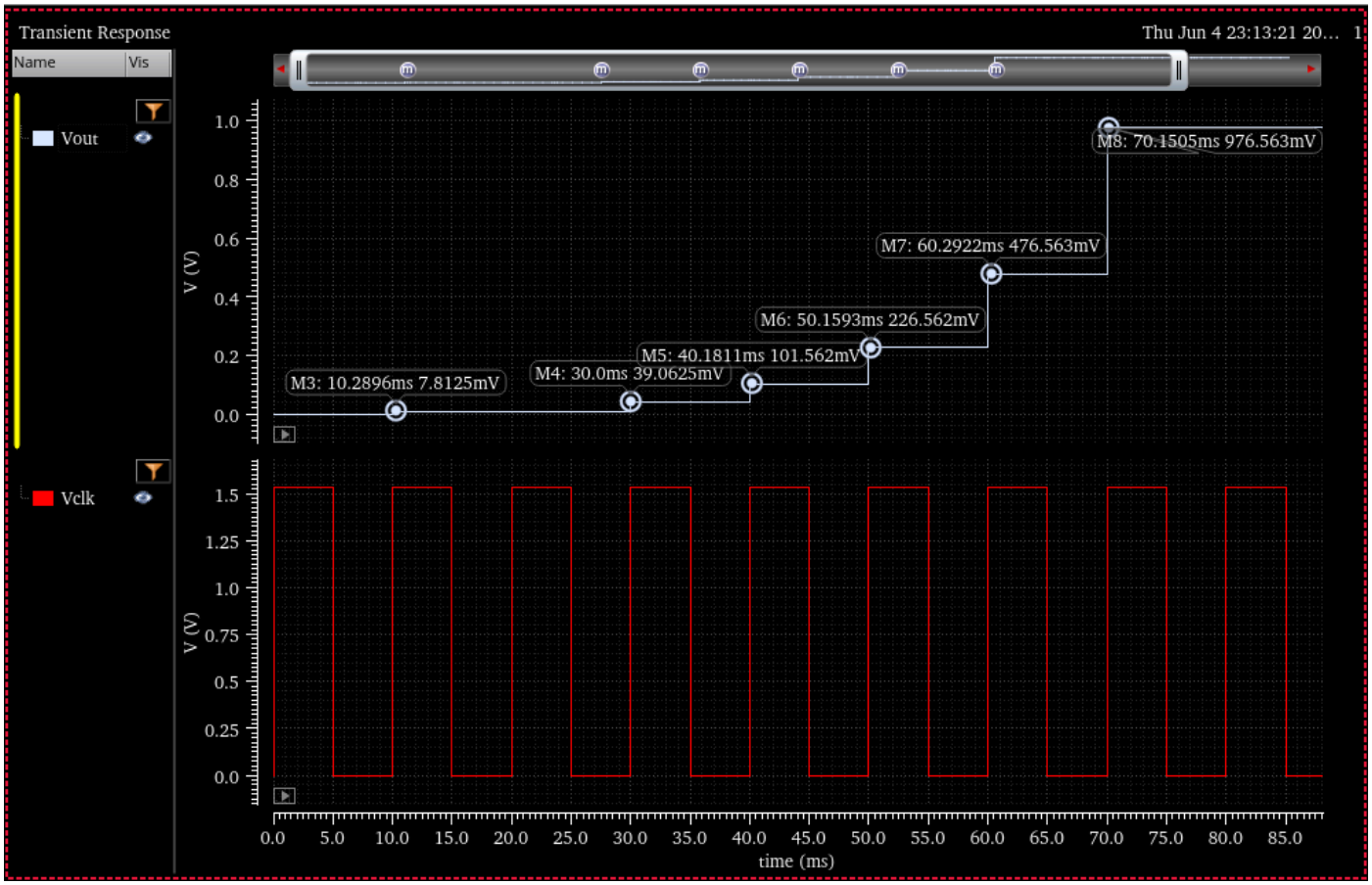
$$V_0 = \frac{2^0}{256} = 3.90625mV,\ V_2 = \frac{2^0+2^2}{256} = \frac{5}{256} = 19.5313mV$$

כלומר הערכים אכן תואמים לערכים התיאורתיים אותם רצינו לקבל, כפי שניתן לראות מהחישוב ומהגרף, והתקבלו תוצאות צפויות מהרכיב.

2) עבור כניסת המשלים ל 5 - 11111010 (250)



תוצאות:



ניתן לראות מהגרף, כצפוי, שמתרחשת עליה במוצא (בלבן) בעליות השעון (באדום) השניה, רביעית, חמישית, שישית, שביעית ושמינית, כלומר המתאימות לקריאת הביטים במקומות 1,3,4,5,6,7, שהן למעשה היחידות שנמצאות ב-1 לוגי במקרה זה.
כעת נבדוק האם ההפרש הדיפרנציאלי בין המוצאים תואם את הערך אנו רוצים לקבל, כלומר לקבל את מספר הקבוצה מנורמל ל256 ביטים. נראה אם החישוב אכן תואם לערכים בגרף:

$$V_1 = \frac{2^1}{256} = 7.8125mV,\ V_3 = \frac{2^1+2^3}{256} = \frac{10}{256} = 39.0625mV$$

$$V_4 = \frac{2^1+2^3+2^4}{256} = \frac{26}{256} = 101.5625mV,\ V_5 = \frac{2^1+2^3+2^4+2^5}{256} = \frac{58}{256} = 226.562mV$$

$$V_6 = \frac{2^1+2^3+2^4+2^5+2^6}{256} = \frac{122}{256} = 476.5625mV$$

$$V_7 = \frac{2^1+2^3+2^4+2^5+2^6+2^7}{256} = \frac{221}{256} = 976.563mV$$

כלומר הערכים אכן תואמים לערכים התיאורתים אותם רצינו לקבל, כפי שניתן לראות מהחישוב ומהגרף, והתקבלו תוצאות צפויות מהרכיב.

חלק 3.2 - רכיב DAC Reference

רכיב ה DAC מקבל אות דיגיטלי וממיר אותו לאות אנלוגי בתצורת reference, כלומר ההמרה שלו כל פעם תוציא מתח רפרנס ולכן הוא דומה לDAC רגיל אך קצת שונה .

ראשית נמשש רכיב COUNTER המבצע ספירה מ0 עד 7 ובכך בכל מחזור שעון נוכל לבצע את ההשוואה בתוך רכיב DAC REF. נבצע זאת כפי שרשום בהנחיות, ע"י המשוואה $code = \text{code} + (2 * \text{comp} - 1) * 2^{7-i}$, כאשר i מייצג את המוצא של הCOUNTER וגם את מספר הכניסה שלנו (את הכניסה ל- DAC REF). ה-comp משווה בין מתחי הכניסה והמוצא, אשר מוגדר פעם כ-'0' ופעם כ-'1' כאשר כאן $V_{DD} = 1V$.
הערך המחושב code הוא ערך המתח השואף למתח המוצא (אחרי נרמול).

3.2.1 - קוד ב-VerilogA עבור הרכיבים

מימוש ה- COUNTER :

```
// VerilogA for HW3_GalAriel, counter, veriloga

`include "constants.vams"
`include "disciplines.vams"

module COUNTER(clk, Reset_b, index);

    input clk, Reset_b;
    output index;

    electrical clk, Reset_b, index;

    integer idx;

    analog begin

        @(initial_step) begin
            idx = 1;
        end


        @(cross(V(Reset_b) - 0.5, -1)) begin
            idx = 1;
        end

        @(cross(V(clk) - 0.5, +1)) begin

            if (V(Reset_b) < 0.5) begin
                idx = 1;
            end else begin

                if (idx < 7)
                    idx = idx + 1;
                else
                    idx = 7;

            end

        end
        V(index) <+ transition(idx, 0, 1n);

    end

endmodule
```

אופן פעולת הקוד :

המודול Counter מממש מונה דיגיטלי פשוט שמייצר את הערך index (משומש במודול של ה DAC REF). בתחילת הסימולציה, בעזרת initial_step, הערך של idx מאותחל ל-1. לאחר מכן, בכל פעם שיש ירידה של Reset_b מתחת ל-0.5, המונה חוזר ל-1 = idx. בנוסף, בכל עלייה של אות השעון clk מעל הסף 0.5, הקוד בודק שוב אם יש reset פעיל. אם הreset אכן פעיל, הוא מאפס את המונה ל-1. אם אין reset, המונה מתקדם באחד בכל מחזור שעון, כל עוד idx < 7. כאשר הוא מגיע ל-7, הוא נשאר שם ולא ממשיך לעלות. בסוף כל שינוי, הערך של idx מועבר ליציאה האנלוגית index בעזרת transition, כך שהמעבר לערך החדש אינו מיידי לגמרי אלא מתבצע בצורה חלקה עם זמן מעבר של 1n.

מימוש DAC REF:

```
// VerilogA for HW3_GalAriel, DAC_ref, veriloga

`include "constants.vams"
`include "disciplines.vams"

module DAC_reference(clk, comp, index, Reset_b, out_ref, vdd, gnd);

    input clk, comp, index, Reset_b, vdd, gnd;
    output out_ref;

    electrical clk, comp, index, Reset_b, out_ref, vdd, gnd;

    real code;
    real VDD;
    real vout;
    integer i;
    integer comp_bit;

    analog begin

        VDD = V(vdd, gnd);

        @(initial_step) begin
            code = 128;
        end


        @(cross(V(Reset_b, gnd) - 0.5, -1)) begin
            code = 128;
        end

        @(cross(V(clk, gnd) - 0.5 * VDD, +1)) begin

            if (V(Reset_b, gnd) < 0.5) begin

                code = 128;

            end else begin

                i = $rtoi(V(index, gnd) + 0.5);

                if (V(comp, gnd) > 0.5)
                    comp_bit = 1;
                else
                    comp_bit = 0;

                code = code + (2 * comp_bit - 1) * pow(2, 7 - i);

                if (code < 0)
                    code = 0;
                else if (code > 255)
                    code = 255;

            end

        end

        vout = VDD * code / 256.0;

        V(out_ref, gnd) <+ transition(vout, 0, 1n);

    end

endmodule

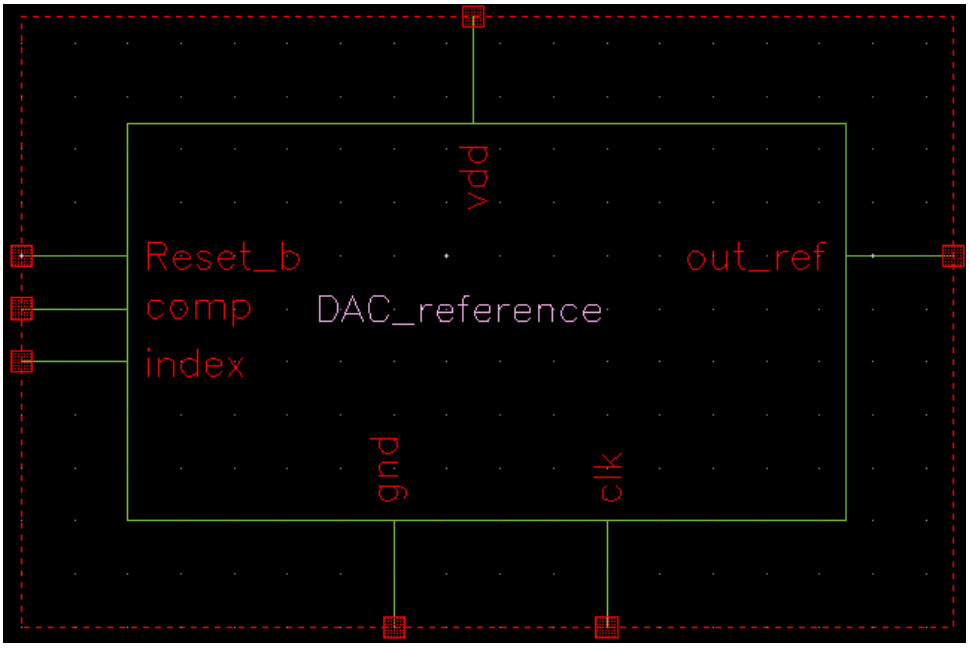
```

אופן פעולת הקוד:

הקוד מממש DAC ייחוס אשר מעדכן את ערך המתח out_ref לפי תוצאת ההשוואה cmp ולפי הערך של index (המוצא של ה-COUNTER). בתחילת הסימולציה הקוד מאתחל את code לערך אמצעי של 128, כלומר בערך חצי מהטווח של DAC בן 8 ביט (בהתאם להנחיה בתרגיל). כאשר אות האיפוס Reset_b יורד מתחת ל־0.5, הקוד מחזיר שוב את code ל־128. בכל עלייה של השעון clk, אם אין reset פעיל, המודול קורא את הערך של index, מחשב לפי זה את הביט הנוכחי שנבדק, ואז משנה את code בהתאם לערך של המשווה cmp. אם comp גבוה, הביט מקבל ערך 1, ואם הוא נמוך, הביט מקבל ערך 0. כלומר הקוד בעצם בונה או מעדכן את מילת ה־DAC לפי החלטות comp. לאחר מכן הקוד נעשית בדיקה האם המוצא code נמצא בטווח התקין של 0 עד 255, ומחושב מתח היציאה לפי vout = VDD * code / 256. לבסוף, המתח הזה מועבר ליציאה out_ref בעזרת transition, כדי שהשינוי במתח יהיה חלק ולא קפיצה מיידית.

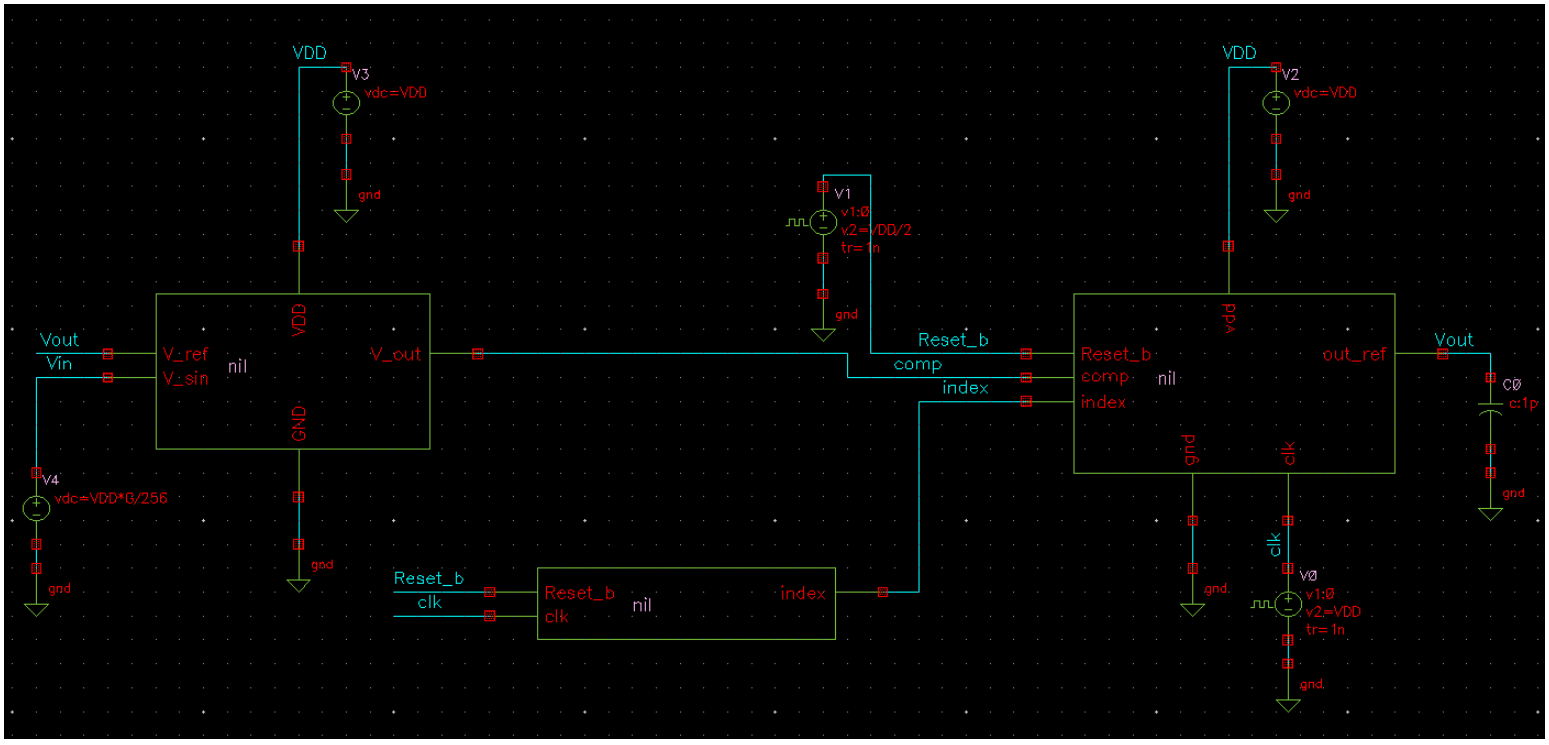
3.2.2 - יצירת סימבול עבור הרכיב

הסימבול:



3.2.3+3.2.4 - Testbench לרכיב

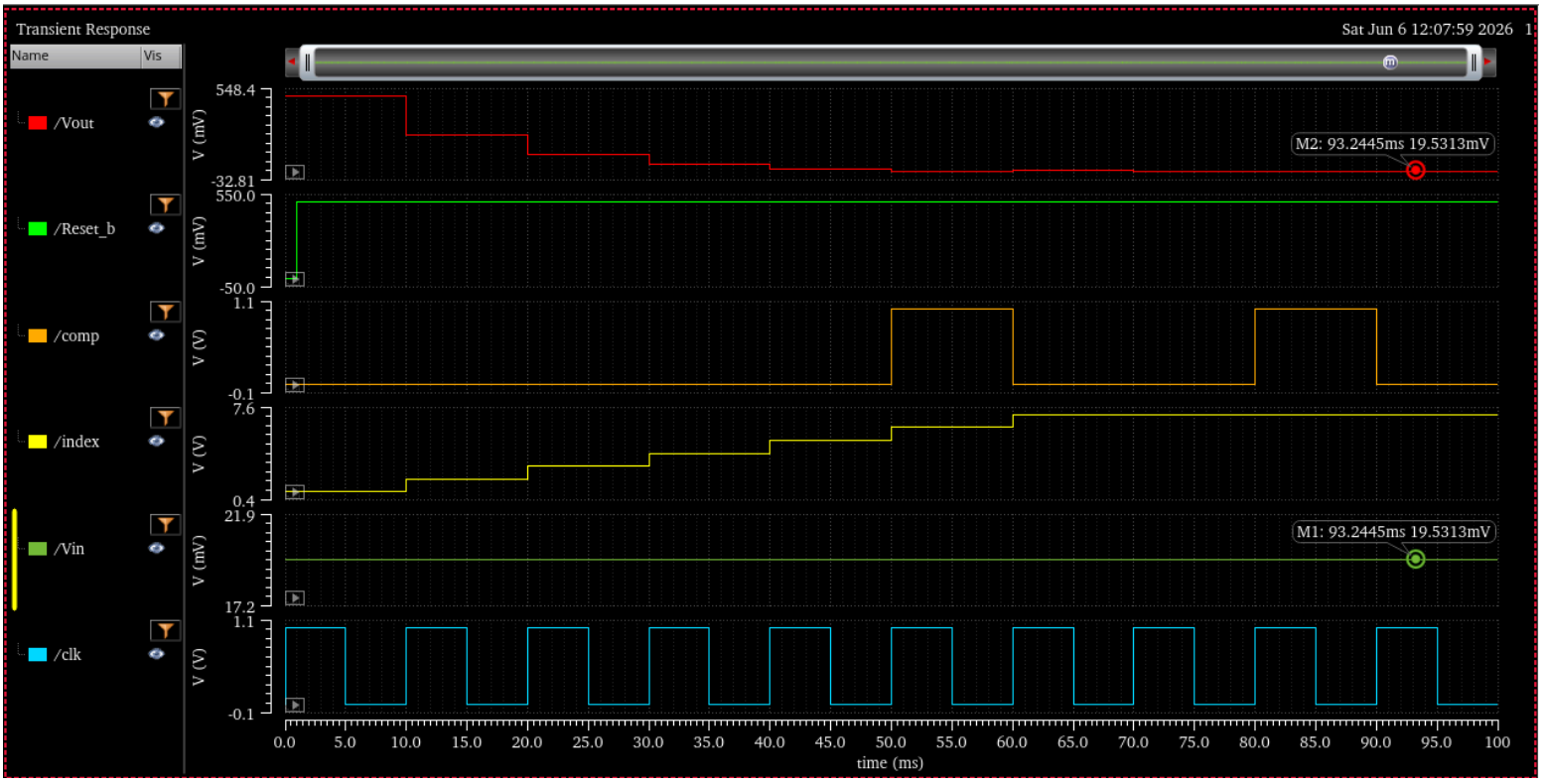
הסכמה לTB:



נגדיר את $V_{in} = V_{DD} * G/256$ ($V_{DD} = 1V$).

כעת נריץ את הסימולציות עבור שני המקרים שתיארנו מעלה, ונדון בתוצאות.

1) עבור כניסה 5 (כמספר הקבוצה שלנו) - 00000101

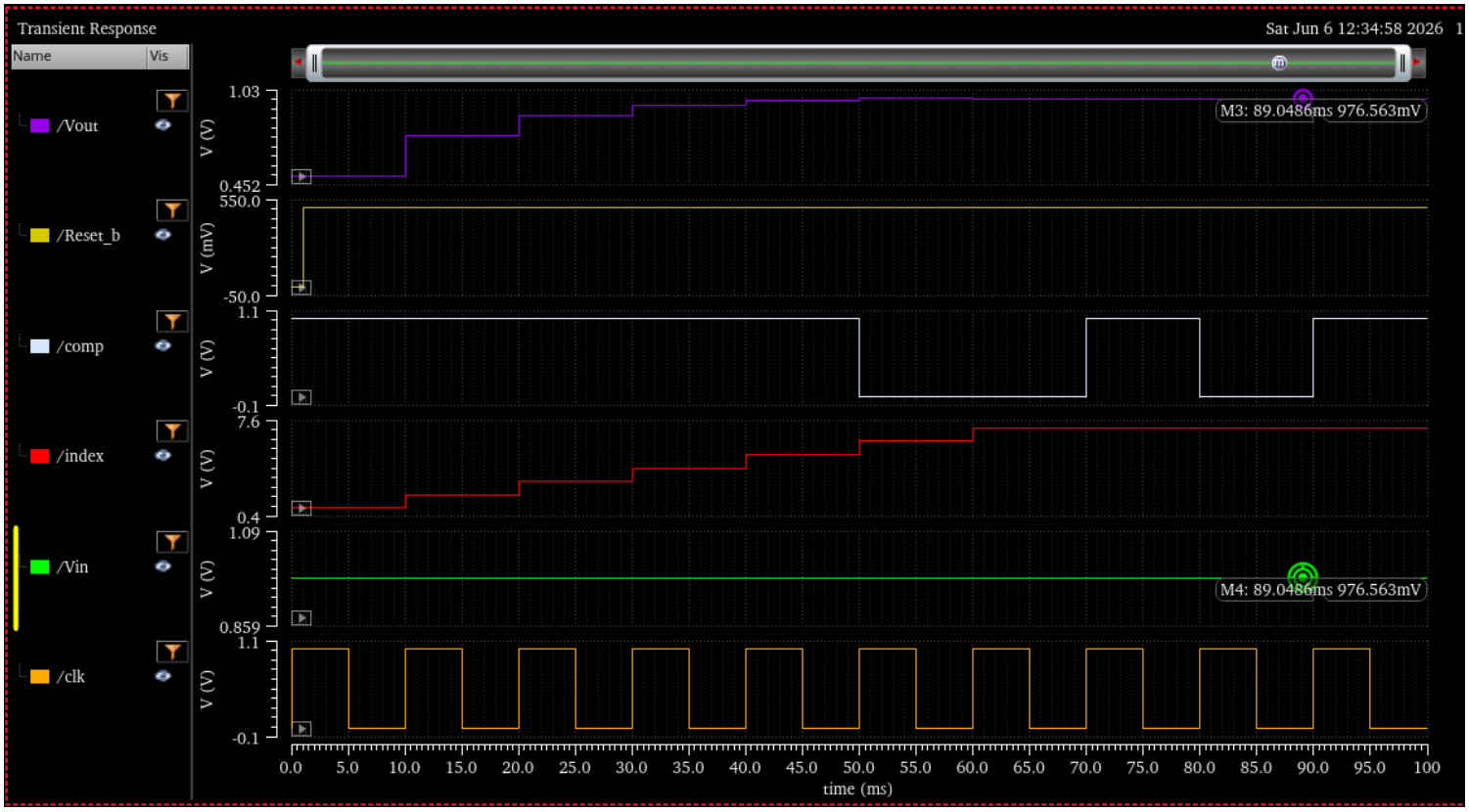


עבור G=5 נקבל מתח כניסה (בירוק בהיר) לפי הנוסחה

$V_{in} = V_{DD} * G/256 = 19.531mV$ ($V_{DD} = 1V$). אות ה־clk (בכחול) הוא זה שקובע מתי מתבצע כל עדכון, ובכל מחזור שעון ה־index (בצהוב) מתקדם לשלב הבא של הבדיקה. (בנוסף מכיוון ש־Reset_b (בירוק) נשאר גבוה, המערכת לא נכנסת לאיפוס בזמן הסימולציה). הקוד הפנימי מאותחל לערך 128 ולכן ערך המוצא ההתחלתי V_{out} (באדום) מאותחל לערך $V_{out} = V_{DD} * \frac{128}{256} = \frac{V_{DD}}{2} = 500mV$.

לפי הערכים שמתקבלים מה- comp (בכתום), ה־DAC משנה את ערך ה־V_{out} בשלבים ובהתאם לנוסחה בתחילת הסעיף, ויורד באופן הדרגתי עד שהוא מתכנס לערך הסופי. במקרה הזה coded שהתקבל הוא 00000101, כלומר 5 בעשרוני, ולכן מתח היציאה הסופי לאחר זמן מה (כלומר כאשר הוא כבר התייצב על ערך סופי) הוא $V_{in} = V_{out} = 19.5313mV$ בהתאם לחישוב בתחילת הפסקה, וכפי שניתן לראות מהגרף. הרכיב פועל כפי שציפינו, ע״י התנהגות שתואמת את הציפיות מרכיב DAC reference בתהליך קירוב הדרגתי.

2) עבור כניסת המשלים ל 5 - 11111010 (250)



עבור G=250 נקבל מתח כניסה (בירוק) לפי הנוסחה

$V_{in} = V_{DD} * G/256 = 976.563mV$, $(V_{DD} = 1V)$. גם כאן הקוד הפנימי מאותחל לערך 128 ולכן ערך המוצא ההתחלתי V_{out} (בסגול) הוא מאותחל לערך

$V_{out} = V_{DD} * \frac{128}{256} = \frac{V_{DD}}{2} = 500mV$, הנמוך מ-$V_{in}$. לכן במקרה זה ה-DAC משנה את ערך ה-V_{out} בשלבים ובהתאם לנוסחה בתחילת הסעיף, אך הפעם עולה באופן הדרגתי (ולא יורד כמו במקרה הקודם) עד שהוא מתכנס לערך הסופי.

כאן ערכו הסופי הוא הוא $V_{in} = V_{out} = 976.563mV$ בהתאם לחישוב בתחילת הפסקה, וכפי שניתן לראות מהגרף. הרכיב גם במקרה הזה פועל כפי שציפינו, ע"י התנהגות שתואמת את הציפיות מרכיב DAC reference בתהליך קירוב הדרגתי.